L06 Lab Report: Time-Series Forecasting Using Nixtla AutoML

1. Data Preparation and Preprocessing

The dataset used in this lab was obtained from Kaggle and consists of time-series temperature readings collected from IoT devices. The dataset was loaded into a Python environment using the Pandas library for efficient data manipulation and analysis.

Initial inspection of the dataset was performed to identify missing values, inconsistencies, and overall structure. Missing values were handled using forward-fill and backward-fill methods to preserve the temporal continuity of the data, which is critical in time-series analysis (Hyndman & Athanasopoulos, 2021). Outliers were visually inspected using plots and handled where necessary to reduce their impact on model performance.

The noted_date column was converted into a datetime format and sorted chronologically to ensure proper sequencing. Additional time-based features such as hour, day, month, and day of the week were extracted to enhance the dataset and provide more context for the forecasting model.

Normalization techniques were applied to ensure that feature values were within a consistent range, improving model stability and performance. The dataset was then split into training and testing sets using a time-based split (80% training and 20% testing), ensuring that future observations were not used to predict past values, which is a critical requirement in time-series forecasting (Hyndman & Athanasopoulos, 2021).

2. Model Selection and Training (Nixtla AutoML)

Nixtla's AutoML framework was used to perform time-series forecasting. The dataset was formatted according to Nixtla's requirements, where the timestamp column was labeled as ds, the target variable (temperature) as y, and a unique_id was assigned to identify the time series.

Using the Nixtla client, the forecasting model was automatically selected and trained based on the dataset's characteristics. AutoML approaches like Nixtla streamline the model selection process by evaluating multiple algorithms and choosing the most appropriate one for the data, reducing the need for manual tuning (Nixtla, 2024).

Nixtla is particularly effective for time-series forecasting because it can capture trends, seasonality, and temporal dependencies inherent in sequential data. This makes it well-suited for Industrial

Internet of Things (IIoT) applications, where sensor data often exhibits complex patterns over time.

3. Feature Engineering

Feature engineering was applied to improve the predictive performance of the model by incorporating both automated and custom features.

Automated Features

Nixtla automatically leverages temporal relationships such as lag values and time dependencies, which are essential for accurate forecasting.

Custom Features

Two domain-specific features were engineered:

1. **Rolling Mean Temperature (Window = 5)**

   This feature calculates the average temperature over a rolling window of five observations, helping to smooth short-term fluctuations and highlight underlying trends.

2. **Temperature Difference (Rate of Change)**

   This feature measures the change in temperature between consecutive readings, enabling the model to detect sudden environmental changes.

Significance of Features

These features significantly enhance the model's ability to:

- Capture temporal trends and seasonality
- Detect anomalies and abrupt changes
- Improve forecasting accuracy in dynamic IIoT environments

Feature engineering is a critical step in machine learning pipelines, as well-designed features can greatly improve model performance (Goodfellow et al., 2021).

4. Model Evaluation and Cross-Validation

The model's performance was evaluated using standard time-series metrics:

- **Mean Absolute Error (MAE)**
- **Mean Squared Error (MSE)**

MAE provides an average measure of prediction error, while MSE penalizes larger errors more heavily, making it useful for identifying significant deviations (Hyndman & Athanasopoulos, 2021).

To ensure robustness, rolling-origin cross-validation was implemented. This method evaluates the model across multiple time windows by progressively expanding the training dataset and testing on future observations. This approach is more appropriate for time-series data compared to traditional cross-validation techniques (Hyndman & Athanasopoulos, 2021).

Results Summary

The model demonstrated consistent performance across validation windows, indicating good generalization. While the model effectively captured overall trends, minor deviations were observed during sudden temperature fluctuations, which is common in real-world sensor data.

Potential Improvements

- Incorporating additional environmental variables such as humidity
- Increasing the complexity of engineered features
- Fine-tuning model parameters for improved accuracy

5. Generative Modeling with Variational Autoencoder (VAE)

A neural network-based generative model was implemented to simulate a Variational Autoencoder (VAE). VAEs are designed to learn the underlying distribution of data and generate new synthetic samples that resemble the original dataset (Kingma & Welling, 2022).

The model was trained to reconstruct temperature values and capture latent patterns in the data. Synthetic time-series data was then generated and used to augment the original training dataset.

Impact of Generative Modeling

The use of synthetic data provided several benefits:

- Reduced overfitting
- Improved model generalization
- Enhanced performance on irregular patterns

Generative models are particularly valuable in IIoT applications where data may be limited or imbalanced, as they help improve the robustness of machine learning models (Goodfellow et al., 2021).

6. Individual Reflection

This lab provided practical experience in applying time-series forecasting techniques to real-world IIoT data. One of the key lessons learned was the importance of proper data preprocessing, particularly maintaining chronological order and handling missing values appropriately.

The use of Nixtla AutoML demonstrated how automated tools can simplify the model selection process while still delivering strong performance. Additionally, feature engineering highlighted the importance of incorporating domain knowledge to improve predictive accuracy.

The implementation of a generative model introduced the concept of synthetic data augmentation, which is especially useful when working with limited datasets. Overall, this lab strengthened my

understanding of forecasting methods, evaluation techniques, and the role of artificial intelligence in industrial applications.

References

Goodfellow, I., Bengio, Y., & Courville, A. (2021). *Deep learning*. MIT Press.

Hyndman, R. J., & Athanasopoulos, G. (2021). *Forecasting: Principles and practice* (3rd ed.). OTexts. https://otexts.com/fpp3/

Kingma, D. P., & Welling, M. (2022). Auto-encoding variational Bayes. *arXiv preprint arXiv:1312.6114*.

Nixtla. (2024). *Nixtla time-series forecasting documentation*. https://nixtla.io